

Dossier - SAE DEV3.1

Dorfromantik

Sommaire :

- I. Introduction**
- II. Les fonctionnalités du programme**
- III. Structure du programme**
 - a. Mini-diagrammes simplifiés**
 - b. Toutes les classes (MVC)**
- IV. Le calcul du score**
 - a. L'algorithme de détection des poches**
 - b. Diagramme d'activité**
 - c. Gestion du score et comparaison avec les autres joueurs**
- V. Améliorations ergonomiques**
 - a. Wireflow/WireFrame**
 - b. Fonctionnalités en plus**
- VI. Conclusion**
 - a. Globale**
 - b. Personnelles**

I. INTRODUCTION

Le jeu que nous avons développé s'inspire librement de Dorfromantik, un jeu de puzzle où le joueur doit assembler des tuiles hexagonales pour former un paysage harmonieux. Le but du jeu est de créer un paysage cohérent en posant des tuiles représentant différents types de terrains tout en respectant certaines règles d'adjacence. À chaque tour, une tuile est révélée au joueur, qui doit décider de sa position et de son orientation pour maximiser les points obtenus. Le jeu met l'accent sur la stratégie, car chaque décision de placement peut influencer considérablement le score final.

Dans ce jeu, les tuiles sont composées de cinq types de terrains : mer, champ, pré, forêt et montagne. Chaque tuile peut contenir un ou deux terrains, et lorsque deux terrains cohabitent sur une même tuile, ils se partagent les côtés. L'élément central du jeu réside dans l'agrandissement progressif du paysage, où des poches de terrains identiques se forment. Chaque poche de terrain est notée en fonction du nombre de tuiles qui la composent, et les points sont calculés par la somme des carrés du nombre de tuiles présentes dans chaque poche.

Le joueur doit faire attention à la manière dont les tuiles sont placées afin de connecter les terrains similaires, car ces connexions forment des poches, et chaque poche permet de marquer des points. Le programme, développé en Java, repose sur une logique de placement stratégique et un suivi des scores pour chaque joueur. À la fin de la partie, le score est comparé à celui des autres joueurs pour évaluer la performance relative.

Ce jeu est un excellent moyen de tester la capacité à créer une expérience ludique tout en respectant des contraintes algorithmiques. De plus, en utilisant un serveur de base de données pour stocker les séries de tuiles et les scores, nous ajoutons une dimension de compétitivité où les joueurs peuvent se mesurer les uns aux autres sur des parties identiques.

II. DESCRIPTION DES FONCTIONNALITÉS

Accueil et Navigation



Fonctionnalité :

Présente un écran interactif avec les options "Jouer", "Règles", et "Contrôles".

Après un clic sur le bouton "Jouer", l'utilisateur est **redirigé directement** vers le menu des saisons et des séries.

Comment nous avons procédé :

Nous avons implémenté une classe **SplashScreen** qui charge dynamiquement une image de fond et propose des boutons transparents pour les options. La redirection vers le menu des séries est gérée via une interaction asynchrone utilisant un **CountDownLatch**.

Classe et méthodes :

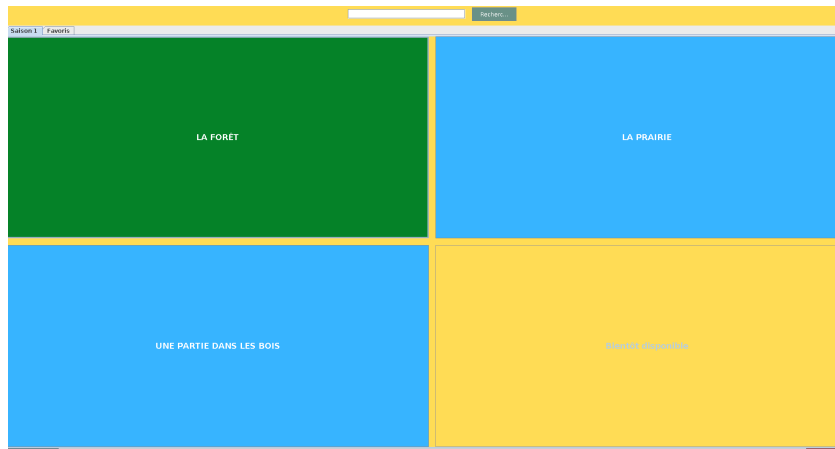
SplashScreen :

- **updateBackgroundImage** : Change dynamiquement l'image de fond.
- **showSplashScreen** : Initialise et affiche l'écran de bienvenue.
- **createTransparentButton** : Créer des boutons transparents pour l'interaction.

Menu principal

Fonctionnalité :

Cette fonctionnalité affiche les séries disponibles ainsi que des options pour quitter ou retourner à l'écran d'accueil. Après avoir sélectionné une série, l'utilisateur est redirigé



vers la partie correspondante. Pour assurer une meilleure maintenabilité à long terme, nous avons structuré le contenu en **saisons**, chacune composée de **plusieurs séries**. L'utilisateur peut choisir une saison en changeant d'onglet situé en haut à gauche. La série la plus récente sera affichée en premier.

Comment nous avons procédé :

Nous avons utilisé des boutons personnalisés (`createRoundedButton`) pour afficher les séries. Chaque bouton est associé à une action spécifique, telle que le lancement d'une série ou l'affichage d'un message pour une série à venir.

Classe et méthodes :

SeriesMenu :

- **showSeriesMenu** : Affiche le menu des séries et initialise les boutons interactifs.
- **createRoundedButton** : Génère des boutons arrondis avec des actions associées.

Gestion des tuiles

Fonctionnalité :

Les tuiles sont générées avec un ou deux terrains et placées sur le plateau selon les règles du jeu.

Comment nous avons procédé :

Nous avons utilisé une classe Tile pour encapsuler les informations des tuiles. Les tuiles sont associées à des terrains spécifiques et peuvent être consultées ou manipulées dynamiquement lors de la partie.

Classe et méthode :

Classe : Tile

- **getTerrainForSide** : Retourne le terrain associé à un côté.
- **hasSingleTerrain** : Indique si une tuile est composée d'un seul terrain.

Génération des tuiles

Fonctionnalité :

Génération aléatoire des tuiles avec un seed prédéfini pour garantir une configuration unique par partie.

Comment nous avons procédé :

La classe TileGenerator génère les mêmes tuiles.

Classe et méthode :

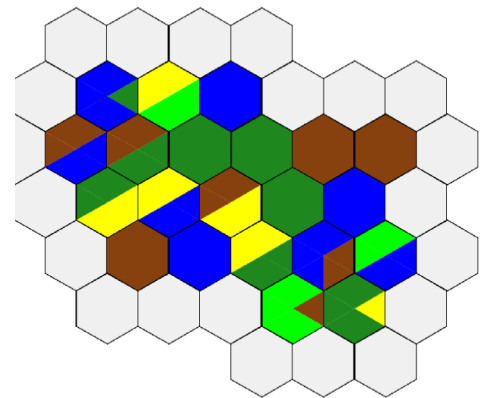
TileGenerator

- **generateRandomTile** : Génère une tuile unique avec un ou deux terrains.
- **isGameFinished** : Vérifie si toutes les tuiles ont été placées.

Affichage du plateau

Fonctionnalité :

Le plateau est composé d'hexagones, composant ainsi la grille du jeu, le plateau est dynamique et a chaque clique sur un hexagone, d'autres hexagones viennent entourer celui-ci.



Comment nous avons procédé :

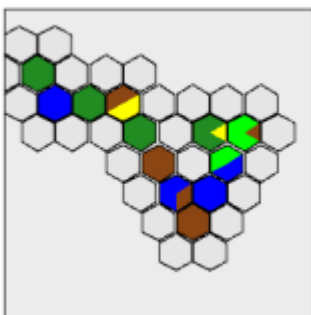
La classe BoardPanel gère l'affichage du plateau et des hexagones. Les mises à jour graphiques sont optimisées avec un tampon pour améliorer les performances.

Classe et méthode :

Classe : BoardPanel :

- **repaintWithNeighbors** : Actualisé graphiquement les hexagones voisins.
- **drawHexagon** : Dessine les hexagones sur le plateau.

Mini-carte



Fonctionnalité :

Pour améliorer le confort du joueur, nous avons intégré une **mini-carte** au projet. Cette fonctionnalité offre une vue d'ensemble du plateau, permettant de suivre l'évolution de la partie facilement.

Comment nous avons procédé :

Nous avons utilisé la classe MiniMapPanel pour afficher une carte réduite, recalculant les proportions dynamiquement selon l'échelle du plateau.

Classe et méthode :

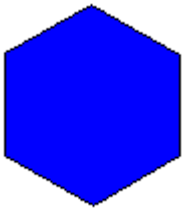
Classe : MiniMapPanel :

- drawMiniHexagon : Dessine les tuiles sur la mini-carte.
- calculateScale : Ajuste l'échelle de la mini-carte en fonction des dimensions du plateau.

Affichage de la tuile suivante

Fonctionnalité :

Une tuile prévisionnelle est affichée en haut à gauche de l'écran. Grâce à elle, le joueur peut savoir quelle prochaine pièce il va poser pour ne pas jouer au hasard.



Comment nous avons procédé :

Nous avons créé un panneau dédié (NextTilePanel) pour afficher et manipuler la tuile suivante.

Classe et méthode :

NextTilePanel :

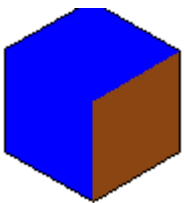
- **rotateRight** et **rotateLeft** : Gèrent la rotation de la tuile.

Interactions Utilisateur et Animations

Rotation de la tuile prévisionnelle

Fonctionnalité :

L'utilisateur peut faire pivoter la tuile prévisionnelle, affichée en haut à gauche de l'écran, à l'aide de la molette de la souris. Cela lui permet de choisir l'orientation de la pièce avant de la placer. Cette solution nous a semblé plus confortable que de devoir placer la pièce, puis ajuster sa rotation.



Comment nous avons procédé :

Un écouteur de la molette (NextTilePanelMouseListener) détecte les mouvements et applique les rotations correspondantes.

Classe et méthode :

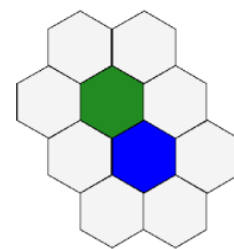
NextTilePanelMouseListener :

- **mouseWheelMoved** : Ajuste la rotation de la tuile selon le sens de défilement.

Placement des tuiles

Fonctionnalité :

Les tuiles peuvent être placées sur le plateau si les règles de placement sont respectées. Les zones qui permettent le placement sont grisées pour une meilleure lisibilité.



Comment nous avons procédé :

La méthode handleMouseClicked dans GameView vérifie la validité de l'emplacement avant de placer la tuile et d'ajouter des voisins.

Classe et méthode :

GameView :

- **handleMouseClicked** : Gère le clic de l'utilisateur pour placer une tuile.

Gestion des Scores

Calcul et enregistrement des scores

Fonctionnalité :

Les scores sont calculés en fonction des correspondances de terrains entre les tuiles adjacentes.

Comment nous avons procédé :

Nous avons centralisé la gestion des scores dans la classe ScoreController, qui interagit avec la base de données pour enregistrer et récupérer les scores.

Classe et méthode :

ScoreController :

- **addScore** : Incrémente le score total.
- **saveFinalScore** : Enregistrer le score final dans la base de données.

Affichage des scores

Fonctionnalité :

Le score courant et les scores environnants sont affichés dynamiquement à l'utilisateur en bas à gauche de la page de jeu, en rouge pour une meilleur visibilité.

Score : 0

Comment nous avons procédé :

Nous avons utilisé ScoreView pour actualiser et afficher les scores après chaque action de l'utilisateur.

Classe et méthode :

ScoreView :

- **updateScore** : Met à jour le score affiché.
- **displaySurroundingScores** : Affiche les scores proches.

Fin de partie

Fonctionnalité :

Une fenêtre affiche le score final, les scores environnants, et compare le résultat au meilleur score, et affiche notre pourcentage de réussite.

Comment nous avons procédé :

La classe ScoreFinal affiche une fenêtre modale contenant les informations de fin de partie et offre des options pour retourner au menu principal.

Classe et méthode :

ScoreFinal :

- **afficher** : Affiche les scores finaux et les options de navigation.



Redirections et Navigation

Modification clé

La redirection après l'écran de bienvenue ou la fin d'une partie est simplifiée pour renvoyer directement au menu des séries (SeriesMenu).

Comment nous avons procédé :

Nous avons utilisé des méthodes de rappel (Runnable) pour garantir une transition fluide entre les écrans, en réutilisant SeriesMenu.showSeriesMenu.

Classe et méthode :

GameView, Main, et SeriesMenu :

- **showSeriesMenu** : Méthode centrale pour afficher les options de séries.

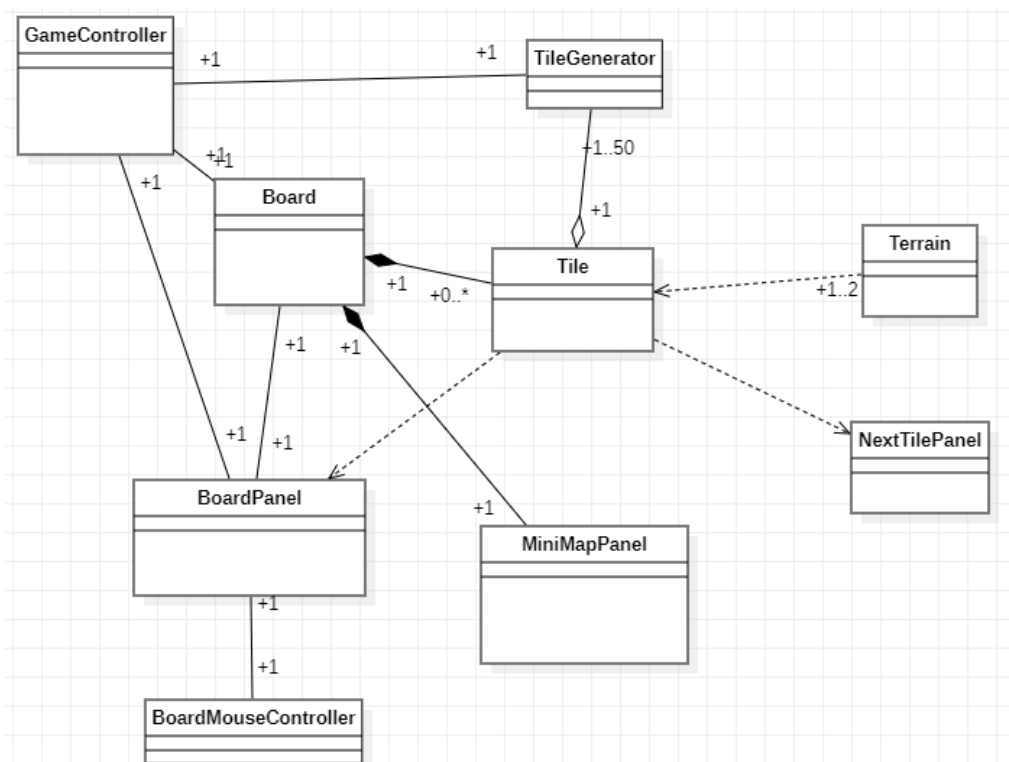
- **scoreFinal.afficher** : Redirige vers le menu après la fin d'une partie.

III. STRUCTURE DU PROGRAMME

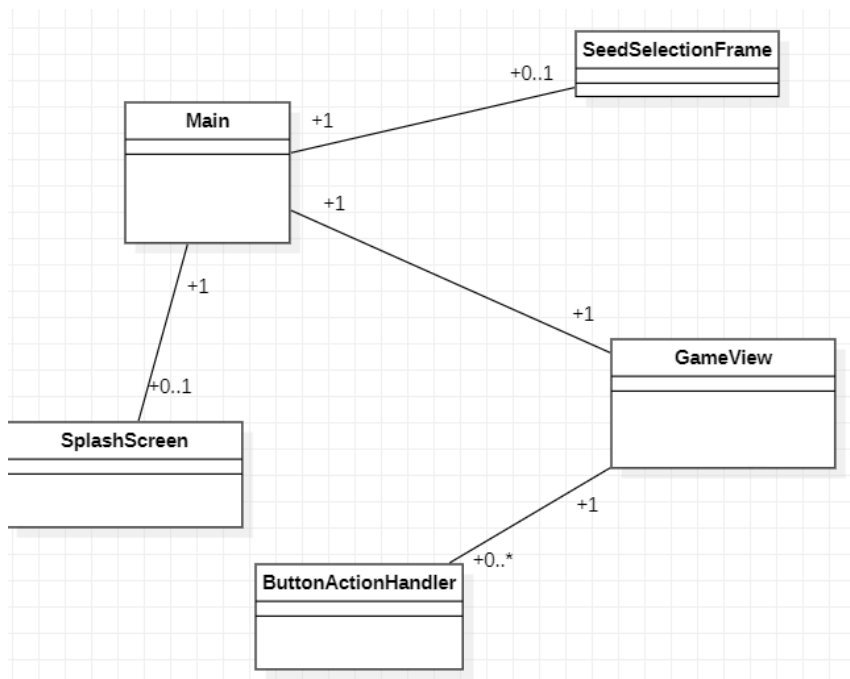
A. Les diagrammes

Par soucis de lisibilité, le diagramme complet du projet à été mis dans le git du projet. Nous avons alors fait des diagrammes plus petits et simplifiés pour décrire les fonctionnalités les plus importantes.

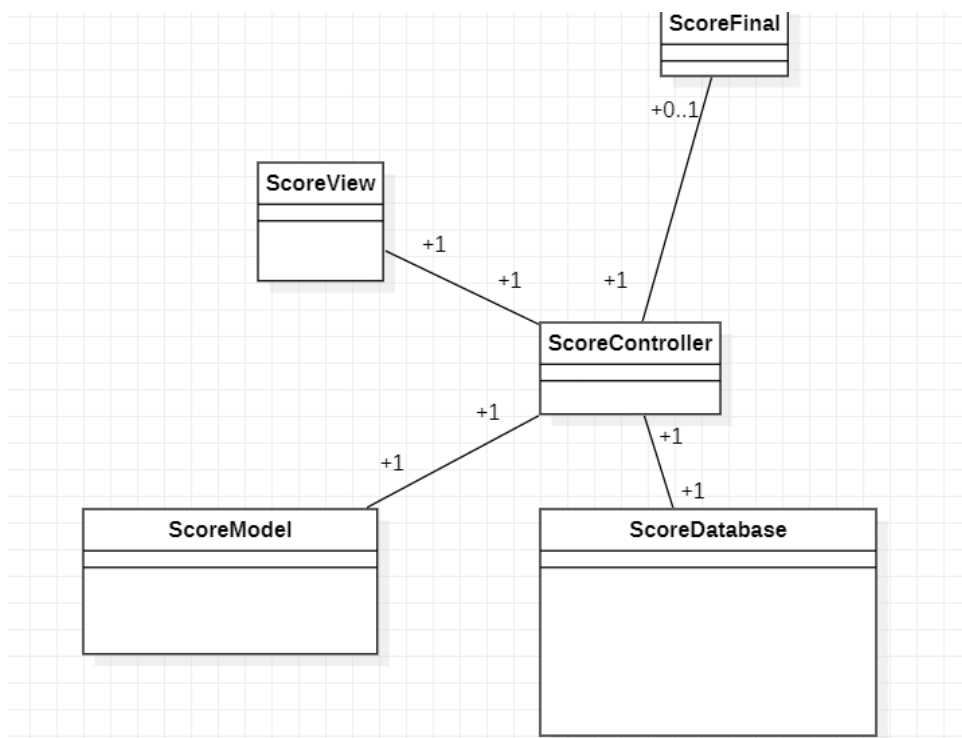
1) Gestion du jeu



2) Gestion des menus



3) Gestion du Score



B. Les Principales classes

1) Models

Tile

Représente une tuile avec un ou deux terrains, ainsi que leurs propriétés (types de terrains, nombre de côtés).

Terrain

Enumération définissant les différents types de terrains disponibles (MER, CHAMP, FORÊT, MONTAGNE, PRÉ).

TileGenerator

Génère des tuiles aléatoires en respectant les règles du jeu et une limite de 50 tuiles par partie.

Board

Représente le plateau de jeu sous forme de grille hexagonale où les tuiles sont placées.

ScoreModel

Stocke et gère le score courant de la partie.

SeedRepository

Interagit avec la base de données pour gérer les seeds (identifiants uniques) des parties.

2) Contrôleurs

ScoreController

Gère l'interaction avec le modèle de score, calcul les scores et les sauvegarde dans une base de données.

GameController

Connecte le plateau (Board) avec la vue du jeu, permettant le démarrage et la gestion principale de la partie.

Main

Point d'entrée du programme, responsable du lancement de l'interface et de la gestion initiale.

HomeAction

Redirige l'utilisateur vers le menu principal.

SwitchToGameAction

Permet de démarrer une partie en fonction d'un seed sélectionné dans le menu.

ButtonActionHandler

Gère les actions des boutons dans le splash screen pour naviguer entre les différentes vues (Menu, Règles, Contrôles).

ResizeListener

Ajuste dynamiquement la disposition des composants graphiques en fonction de la taille de la fenêtre.

MouseClickedListener

Traite les clics de souris pour interagir avec les hexagones du plateau.

BoardMouseListener

Permet de déplacer le plateau en glissant la souris ou d'interagir avec des cases spécifiques.

3) View

GameView

Fournit une interface graphique pour le jeu, y compris le plateau, la mini-carte, les scores, et la gestion des tuiles.

NextTilePanel

Affiche la tuile suivante à placer et permet de la faire pivoter.

BoardPanel

Gère le rendu du plateau de jeu avec les tuiles placées.

MiniMapPanel

Fournit une vue réduite du plateau pour suivre la progression de la partie.

ScoreView

Affiche le score courant ainsi que des scores environnants pour la comparaison.

ScoreFinal

Montre une fenêtre modale avec le score final et compare celui-ci au meilleur score de la série.

SeriesMenu

Propose une interface permettant de choisir parmi différentes séries de parties.

SplashScreen

Présente l'écran d'accueil avec un menu interactif permettant de lancer une partie ou d'accéder aux instructions.

IV. CALCUL DU SCORE

a. Analyse de l'algorithme d'identification des poches

Nous avons réussi à faire qu'une partie du score qui est d'ajouter un certain nombre de points à chaque fois que deux tuiles de la même couleur sont placées côte à côte mais le calcul n'est pas correct lorsque la tuile comporte de terrains. En effet, le problème majeur réside dans la manière dont les côtés sont comparés : le côté nord de la tuile posée est comparé au côté nord de la tuile adjacente. Cela empêche une détection correcte des correspondances.

Néanmoins, nous avons réfléchi à comment le faire. Cette partie servira alors à expliquer la meilleure solution, selon nous, pour arriver à un résultat fonctionnel.

But de l'algorithme :

L'objectif est de détecter des poches, c'est-à-dire des groupes d'hexagones (ou tuiles) qui remplissent certaines conditions spécifiques dans le cadre du jeu. Ces poches peuvent être définies par un ensemble de tuiles adjacentes partageant le type de terrain.

Étapes principales de l'algorithme :

1- Pose d'une pièce

Lorsque l'on pose une pièce, l'algorithme regarde les pièces adjacentes à celle-ci.

Dans le programme :

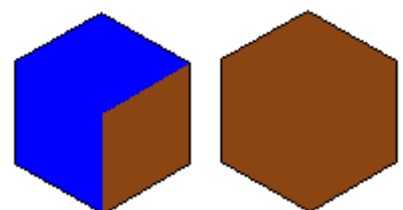
Une méthode pose la pièce et une autre récupère toutes celles qui sont adjacentes.

2-Détection des terrains

Le programme va regarder si le terrain du côté d'une pièce correspond au terrain du côté opposé de la pièce d'à côté.

les côtés sont rangé comment tel :

- sud-ouest : océan
- ouest : océan
- nord-ouest : océan
- nord-est : océan
- est : terre
- sud-est : terre



Dans le programme :

On regarde le terrain qui correspond au côté est de la pièce que l'on place (pièce 1). Si il y a une pièce déjà posée à droite de la pièce que l'on veut posée (pièce 2), on regarde alors le côté OUEST de celle-ci. Si le côté EST de la pièce que l'on vient de poser est du même terrain que le côté OUEST de la pièce déjà posée comme sur l'image, alors on crée une poche.

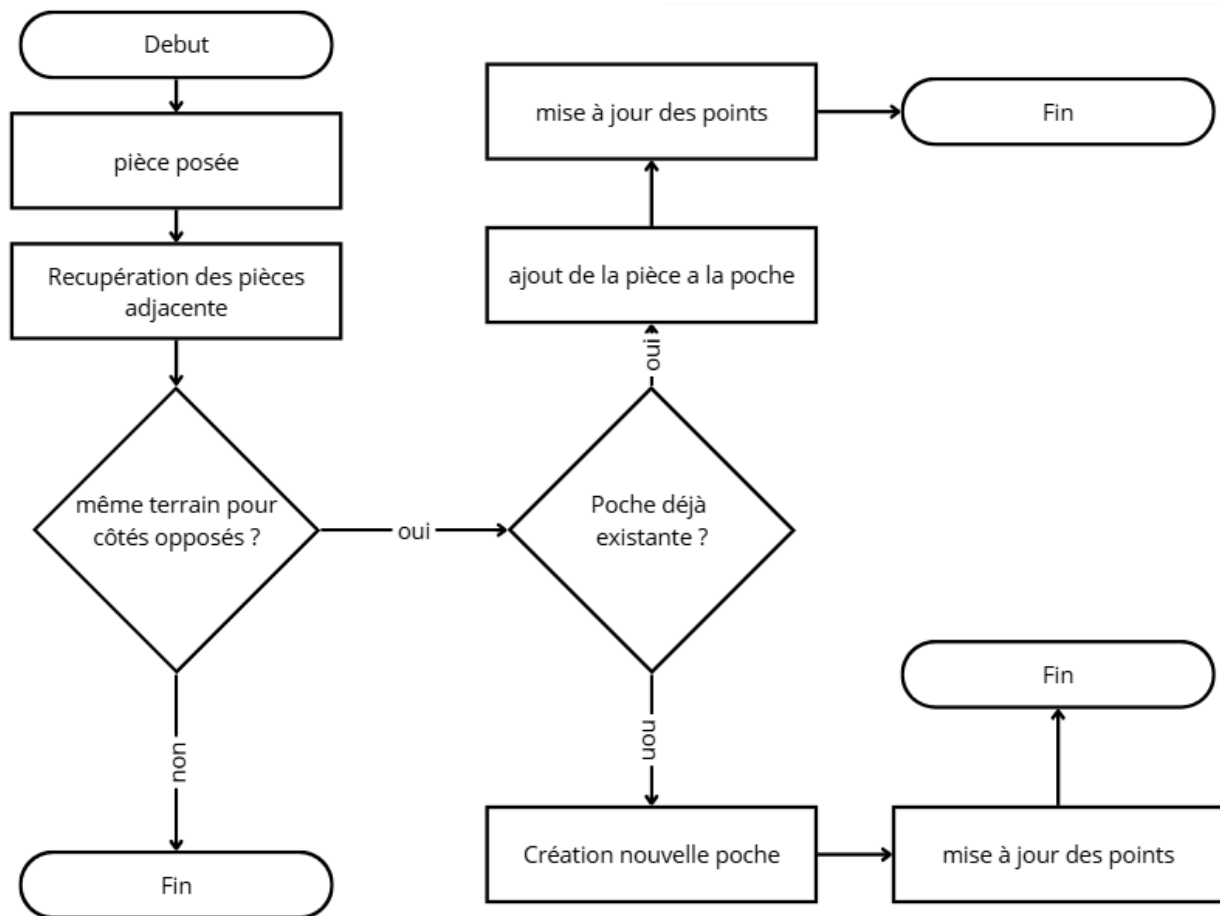
3- Superposition des poches :

Le programme ne doit pas créer de poche tout le temps. Certains critères doivent être respectés. Les côtés doivent être bons (voir étape 2). Et il ne doit pas y avoir de poche déjà présente. Par exemple, on pourrait ajouter une pièce avec des côtés qui sont bons, mais si les pièces adjacentes sont déjà dans une poche correspondante, alors la nouvelle pièce doit s'ajouter à la poche déjà existante et ne doit pas en créer une nouvelle.

Dans le programme :

Lorsque l'on pose une pièce et que celle-ci semble vouloir créer une nouvelle poche, on regarde s'il n'y avait pas une poche déjà existante. Si c'est le cas, on ajoute les terrains de la nouvelle pièce à la poche et on augmente les points en fonction.

b. Diagramme d'activité



c. Gestion du score et comparaison avec les autres joueurs

Fonctionnalité :

L'affichage final du score récapitule la performance de l'utilisateur en montrant :

- Le score final.
- Une comparaison avec le meilleur score.
- Une liste des scores environnants.
- Le pourcentage de réussite.
- Une option pour retourner au menu principal.

Comment nous avons procédé :

Sauvegarde et récupération des scores : Le score final est enregistré et comparé via `saveFinalScore()` et `getBestScoreForSeries()`. Les scores environnants sont récupérés dynamiquement avec `getSurroundingScoresForDisplay()`.

Affichage graphique : La classe `ScoreFinal` utilise des panneaux (`JPanel`), labels (`JLabel`), et couleurs pour afficher les informations.

Navigation et interactions : Un bouton permet de revenir au menu principal via un callback.

Classe et méthodes :

Classe : `ScoreFinal`

- `afficher()` : Affiche l'écran final, organise le score, les comparaisons, et les options.

Classe : `ScoreController`

- `saveFinalScore()` : Sauvegarde le score final.
- `getBestScoreForSeries()` : Récupère le meilleur score.
- `getSurroundingScoresForDisplay()` : Fournit les scores environnants.
- `getTotalScore()` : Donne le score final.

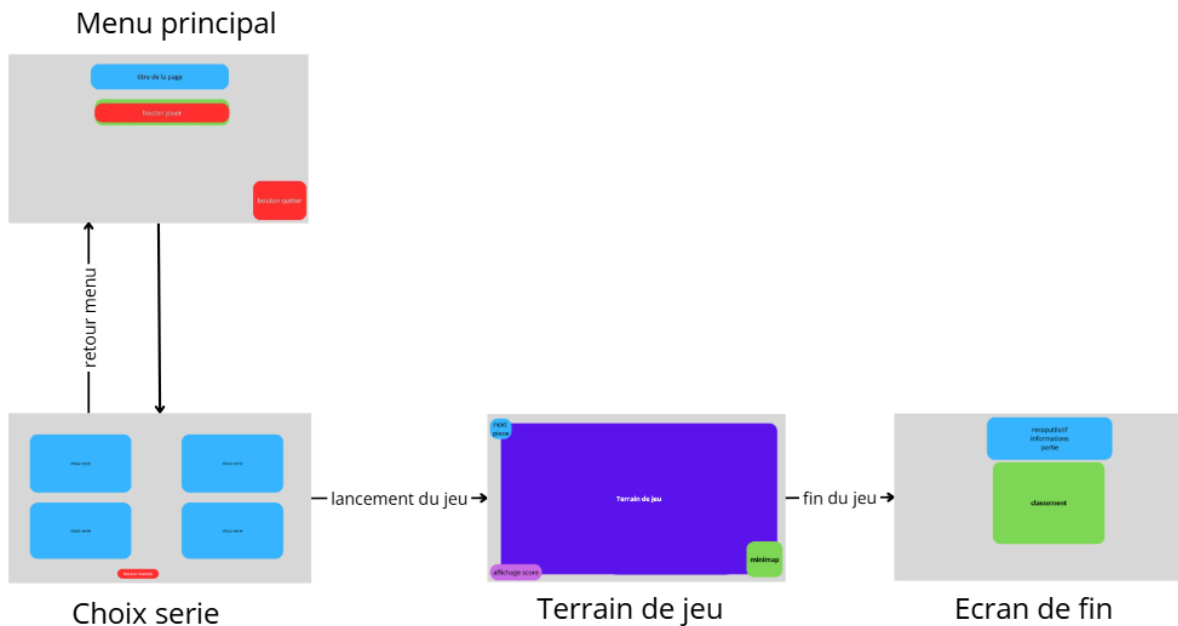
Classe : `ScoreView`

- `updateScore()` : Met à jour l'affichage du score en cours de partie.
- `displaySurroundingScores()` : Affiche les scores environnants avec surbrillance.

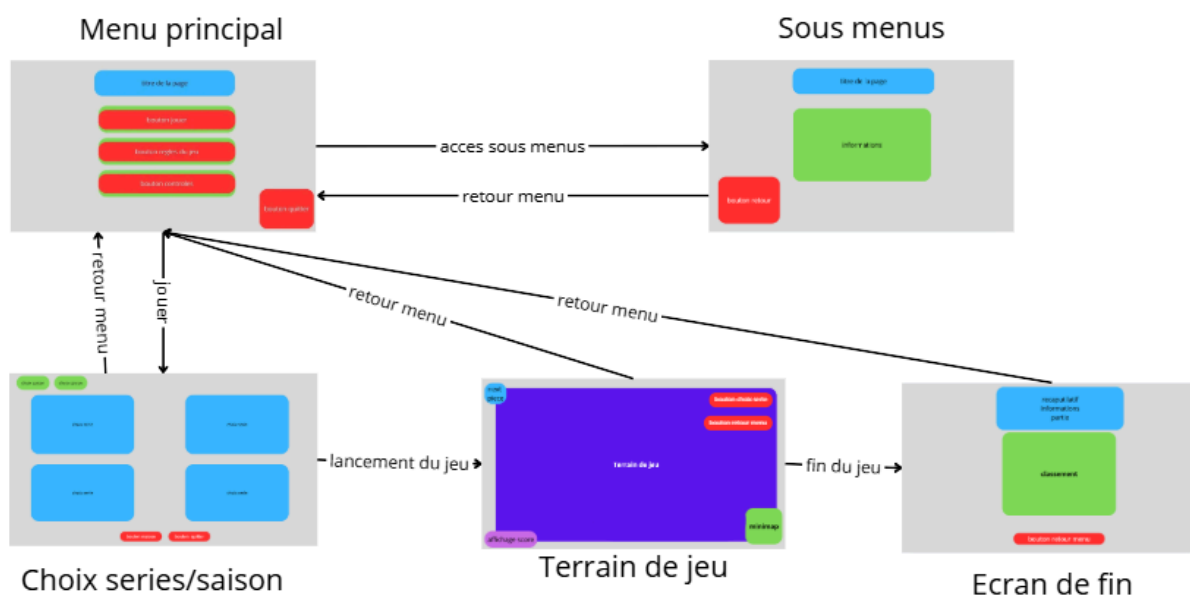
V. AMÉLIORATIONS ERGONOMIQUES

a. WireFlow / WireFrame

Dès le départ, grâce aux wireframes, nous avons une vision claire, mais nous avons ajusté les pages pour améliorer l'ergonomie et répondre aux attentes des utilisateurs.



WireFlow de début de projet



WireFlow de fin de projet

b. Fonctionnalités supplémentaires

Nous avons donc ajouté des éléments visibles pour enrichir l'expérience utilisateur :

- **Des boutons de retour au menu** dans toutes les pages pour une meilleure navigation.
- **Une mini-carte** dans le jeu pour apporter une meilleure visibilité du plateau au joueur.
- **Des choix de saisons en plus des séries** pour une meilleure maintenabilité future.
- **Une barre de recherche** pour trouver facilement la série de jeux souhaitée.
- **De nouvelles pages dans les menus** pour expliquer les règles et les contrôles.
- **Le déplacement du plateau de jeu** grâce aux flèches directionnelles ou avec un clic droit.

Ajouts envisageables

Dans une logique d'amélioration, nous avons pensé à certaines mises à jour qui pourraient grandement améliorer le jeu :

1. La possibilité de mettre des séries en favoris pour permettre aux joueurs de retrouver leurs séries préférées rapidement.
2. La mise en place d'un chronomètre pour les joueurs les plus aguerris.
3. Un (ou plusieurs) terrain(s) qui donnent plus de points.
4. La mise en place de séries plus difficiles que d'autres (avec une difficulté sélectionnable).

CONCLUSION

a. Globale

À la fin de ce projet, nous sommes fiers d'avoir conçu et développé un jeu inspiré de **Dorfromantik** en Java, répondant aux attentes de notre cours. Ce projet a été une aventure collective, où chacun a pu explorer ses forces : certains ont pris en main la création de l'interface graphique, d'autres se sont plongés dans la logique du jeu ou encore dans la gestion des données.

Grâce à une organisation réfléchie et une collaboration fluide, nous avons réussi à donner vie à un jeu à la fois fonctionnel et captivant, tout en respectant les délais. Ce projet a été bien plus qu'un exercice technique : il nous a permis de consolider nos compétences en Java, de vivre une vraie expérience de travail d'équipe et de découvrir les multiples facettes de la conception d'un jeu vidéo.

De la génération des tuiles à la gestion des scores en passant par les interactions avec l'utilisateur, chaque étape a été un apprentissage enrichissant. Ce projet nous a donné un aperçu concret de ce qu'implique la création d'un produit ludique et interactif, et nous en ressortons grandis, prêts à relever de nouveaux défis.

b. Personnelles

Djedjiga :

Ce projet m'a appris beaucoup de choses sur le plan technique, notamment sur l'exploitation de bases de données pour le jeu, ce qui a été très enrichissant.

Ce qui m'a marqué le plus reste le travail sur l'ergonomie, en particulier pour l'affichage des séries. Ce sujet m'a vraiment poussé à réfléchir à la meilleure façon de présenter les choses pour que ce soit intuitif et agréable pour le joueur. J'ai essayé plusieurs versions, discuté avec l'équipe et pris en compte leurs retours. J'ai beaucoup aimé cette partie du projet, car elle demande de se mettre à la place de l'utilisateur et de penser à son expérience.

En résumé, ce projet m'a apporté des compétences techniques et m'a aussi fait découvrir le plaisir de concevoir des interfaces ergonomiques.

Marwa :

Ce projet Dorfromantik a permis de concevoir une architecture logicielle et graphique cohérente, assurant une expérience de jeu fluide et intuitive.

Sur le plan de l'ergonomie, j'ai particulièrement réfléchi à l'expérience du joueur, en me mettant à sa place pour anticiper ses besoins et faciliter l'interaction avec le plateau. J'ai notamment suggéré la possibilité de déplacer virtuellement le plateau à l'aide des touches fléchées, afin que la navigation soit plus naturelle, ainsi que l'ajout d'une mini-map pour offrir une vue d'ensemble et aider le joueur à repérer facilement sa position. J'ai discuté de ces idées avec le reste du groupe, afin de valider leur pertinence.

Ensemble, nous avons conclu que ces améliorations rendraient l'interface plus pratique et plus agréable à utiliser, et nous avons donc implémenté ces fonctionnalités. Ce travail collectif sur l'ergonomie, centré sur les besoins réels du joueur, a permis de rendre l'expérience de jeu plus accessible, claire et confortable.

Raphael :

Ce projet a été une expérience enrichissante tant sur le plan technique que collaboratif. J'ai pu appliquer mes connaissances en Java pour créer un jeu fonctionnel. Travailler en groupe m'a permis de mieux comprendre l'importance de la répartition des tâches et de la communication dans un projet. Cette expérience a renforcé mes compétences en programmation, mais m'a aussi ouvert l'esprit sur les défis de développement d'un jeu, en particulier pour garantir une expérience utilisateur fluide et immersive. Ce projet m'a motivé à continuer d'explorer les différents aspects du développement logiciel.